

A Bounded Binary64 Exponential

LeanExe library PoC

September 2026

1 Task and numerical domain

The task is to approximate e^x on $[-1, 0]$. This implementation uses the degree-six Taylor polynomial

$$p_6(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120} + \frac{x^6}{720}.$$

The exponential power series appears in DLMF equation 4.2.19 [1]. On this interval its alternating tail has magnitude at most $1/5040$ in exact arithmetic. The target bound for the binary64 computation is $1/4000$.

`expTaylor6` accepts and returns binary64 bit patterns in `UInt64` words. The result is optional: a finite input in $[-1, 0]$ returns a value, and other inputs return `none`. Both signed zeros return the exact bits of one. Negative subnormals are admitted. Infinities and NaNs are rejected by the bit-range check.

2 Implementation and coefficient choices

```
import LeanExe.Float64

namespace LeanExe.Lib.Transcendental

def expTaylor6Polynomial (x : UInt64) : UInt64 :=
  let a := Float64.addBits (Float64.mulBits 0x3F56C16C16C16C17 x) 0x3F81111111111111
  let b := Float64.addBits (Float64.mulBits a x) 0x3FA5555555555555
  let c := Float64.addBits (Float64.mulBits b x) 0x3FC5555555555555
  let d := Float64.addBits (Float64.mulBits c x) 0x3FE0000000000000
  let e := Float64.addBits (Float64.mulBits d x) 0x3FF0000000000000
  Float64.addBits (Float64.mulBits e x) 0x3FF0000000000000

def expTaylor6 (bits : UInt64) : Option UInt64 :=
  if bits == 0 || (bits >= 0x8000000000000000 && bits <= 0xBFF0000000000000) then
    some (expTaylor6Polynomial bits)
  else none

end LeanExe.Lib.Transcendental
```

The listing is read from the executable source. Each multiplication and addition rounds separately. The hexadecimal literals record the binary64 coefficient values and keep decimal conversion out of the component. `expTaylor6Polynomial` exposes the unguarded evaluator. `expTaylor6` applies the domain guard before calling it. The complete polynomial uses six multiplications and six additions and allocates no arrays.

This implementation follows `Project.ExpSmall.Model`: its coefficient words, evaluation order, and domain are the same. The earlier numerical and execution development proves the $1/4000$ bound for that model [2]. The new library uses `LeanExe.Float64` primitives, and its connection to the earlier theorem is deferred. The tests below provide evidence for the new integration on their specified inputs.

3 A decay client

For dimensionless time $t \in [0, 1]$, the decay client returns an approximation to the remaining fraction e^{-t} . It checks t , negates its binary64 sign bit, and calls the exponential component. The calculation corresponds to unit initial quantity and time measured in units of the decay time constant.

```
tools/seminum exp -0.5
0.6065321180555556
tools/seminum decay 0.5
0.6065321180555556
tools/seminum exp-bits bfe0000000000000
3fe368b60b60b60c
```

The decimal commands convert their inputs to binary64 in the host and display decimal representations of returned bits. The hexadecimal command preserves the input and output representations. Arithmetic runs in the compiled WASM. Out-of-domain commands exit with status 2 and a diagnostic on standard error.

4 Tests and next comparisons

The execution test compares Wasmtime and native Lean bit results on a uniform interval sample, both signed zeros, subnormals, neighboring boundary words, infinities, and NaNs. Accepted outputs are finite and positive in these tests. The test also compares them with the host exponential. The largest sampled absolute difference is about $1.7611438411335723 \times 10^{-4}$ at $x = -1$, within the target $1/4000$. The decay client receives corresponding domain and rejection tests.

Further implementations could change the polynomial, evaluation method, or range reduction. Each would state its own domain, error target, and operation order. Their catalog entries would share the exponential task while recording their particular algorithm and implementation.

References

- [1] NIST Digital Library of Mathematical Functions, equation 4.2.19. <https://dlmf.nist.gov/4.2.E19>. The power series supplies the coefficients before rounding and truncation.
- [2] LeanExe, *Verified numerical WASM demonstrations*, section “Small exponential.” The source and proofs are in the repository’s `Project.ExpSmall` modules. This development supplies the coefficient words, operation order, and existing model theorem. The new library integration has separate execution tests.